claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_ae3ec722-12a\agent-afe8a8fe47d216faf.jsonl`
- SHA-256: `ea4bc6d25706c40e577ffb6ae8584baf2efbed5e59fde08065ccb517808639d6`
- Source modified: `2026-07-06T07:55:14+00:00`
- Imported at: `2026-07-08T16:00:08+00:00`
- project: `wf_ae3ec722-12a`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T07:52:24.502Z)

```
Reviewing GitHub PR #37 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-c-
delivery-routing -> main, one commit 100abb2).
It implements ADR 0011 Track C (docs/adr/0011-provider-neutral-domain-schema.md): populates &
uses the
destinations / routing_rules / delivery_attempts tables that Track B (#35, merged) created
empty.
Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/delivery.py,
src/tg_video_filter/models.py,
tests/test_db.py, tests/test_delivery.py. Diff +675/-9.

Key new code:
- db.py: _migration_004_destinations_and_routing (backfills destinations from
filter_config_deprecated,
  routing_rules per source, delivery_attempts from videos_deprecated.forwarded=1); repos
upsert_destination,
  get_destination, upsert_delivery_attempt, record_delivery (UPSERT with attempt_count
increment), get_delivery_attempt.
- delivery.py: forward_to_target/send_link_message/deliver gain an optional destination_id
kwarg; when set,
  record_delivery writes a DeliveryAttempt; when None (all current live callers) falls back to
the mark_forwarded no-op.
- models.py: Destination, RoutingRule, DeliveryStatus, DeliveryAttempt.
Note: routing is NOT yet wired into the live path (telethon_client/manager still call deliver()
with destination_id=None);
destination_id is currently exercised only by tests. So record_delivery issues are LATENT until
Track D, but ship now.

Ground rules:
- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run a repro: C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. Build
schema via db._apply_schema(conn) on aiosqlite ":memory:";
  or db.init_db(path) on a temp file seeded with a legacy v1 schema
(users/channels/videos/filter_config)
  to exercise the migration ladder (001->004). NOTE: a legacy 'videos' table MUST include
columns
  duration_s,width,height,size_bytes,mime_type or migration 002's column-copy fails (that's a
test-harness
  requirement, not a bug).
- Authoritative local gate ALREADY RUN (don't re-run full suite): ruff check clean; pytest 451
passed @ 95.75%
  (floor 80%); ruff format --check flags commands.py ONLY because this branch predates the
merged #36 format
  fix (commands.py is NOT in this PR) ? do NOT report that as a #37 finding.
```

```
- Report ONLY defects in THIS PR's diff. Distinguish LIVE-NOW (migration runs on upgrade) from
LATENT
  (delivery recording, unused until Track D). No praise. Concrete failure scenario per finding.
Rank severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour on upgrade), high (real bug,
narrow trigger),
  medium (latent-but-real logic bug), low (hygiene/edge). Be honest; downgrade unrealistic
triggers.

DIMENSION: test quality & coverage gaps (tests/test_db.py, tests/test_delivery.py).
Investigate:
- upsert_delivery_attempt is defined + exported in __all__ but appears to have NO caller and NO
test
  (db.py ~1324-1355 uncovered). Confirm it's dead/untested.
- The destination_id + FAILURE recording path (record_delivery success=False through deliver) ?
is it tested?
  (only success is tested in test_deliver_with_destination_id_records_delivery_attempt.)
- BOTH-mode + destination_id (forward ok / link fail) ? is the masking behaviour tested? (it is
not).
- Migration 004 tests: do they cover the 'me' default (no delivery_target), the no-filter_config
account,
  and multi-source/multi-account cases? Coverage regressed 96.5%->95.75%.
Return findings per schema. A missing critical test IS a finding (severity by the bug it would
have caught).
```

2. assistant (2026-07-06T07:52:27.034Z)

I'll review PR #37 focusing on test quality and coverage gaps. Let me start by reading the key files.

3. user (2026-07-06T07:52:28.032Z)

```
1280               VALUES (?, ?, ?, ?, ?, ?, ?)
1281               ON CONFLICT(account_id, provider, target) DO UPDATE SET
1282                 kind=excluded.kind,
1283                 title=excluded.title,
1284                 is_output=excluded.is_output""",
1285            (
1286                dest.account_id,
1287                dest.provider,
1288                dest.target,
1289                dest.kind,
1290                dest.title,
1291                int(dest.is_output),
1292                dest.created_at.isoformat(),
1293            ),
1294        )
1295        await conn.commit()
1296        conn.row_factory = aiosqlite.Row
1297        async with conn.execute(
1298            """SELECT * FROM destinations
1299               WHERE account_id = ? AND provider = ? AND target = ?""",
1300            (dest.account_id, dest.provider, dest.target),
1301        ) as cur:
1302            row = await cur.fetchone()
1303        assert row is not None
1304        return _row_to_destination(row)
1305
1306
1307    async def get_destination(
1308        conn: aiosqlite.Connection, account_id: int, provider: str, target: str
1309    ) -> Destination | None:
1310        """Resolve a destination by its (account, provider, target) natural key."""
1311        conn.row_factory = aiosqlite.Row
1312        async with conn.execute(
```

```
            "SELECT * FROM destinations WHERE account_id = ? AND provider = ? AND target =
?",
            (account_id, provider, target),
        ) as cur:
            row = await cur.fetchone()
        return _row_to_destination(row) if row else None


    async def upsert_delivery_attempt(
        conn: aiosqlite.Connection, attempt: DeliveryAttempt
    ) -> DeliveryAttempt:
        """Insert or update a delivery_attempt; idempotent on (content_item,
destination)."""
        await conn.execute(
            """INSERT INTO delivery_attempts
                 (content_item_id, destination_id, route_id, status, attempt_count,
                  last_error, created_at, delivered_at, next_retry_at)
               VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
               ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
                 status=excluded.status,
                 attempt_count=excluded.attempt_count,
                 last_error=excluded.last_error,
                 delivered_at=excluded.delivered_at,
                 next_retry_at=excluded.next_retry_at""",
            (
                attempt.content_item_id,
                attempt.destination_id,
                attempt.route_id,
                attempt.status.value,
                attempt.attempt_count,
                attempt.last_error,
                attempt.created_at.isoformat(),
                attempt.delivered_at.isoformat() if attempt.delivered_at else None,
                attempt.next_retry_at.isoformat() if attempt.next_retry_at else None,
            ),
        )
        await conn.commit()
        conn.row_factory = aiosqlite.Row
        async with conn.execute(
            "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id =
?",
            (attempt.content_item_id, attempt.destination_id),
        ) as cur:
            row = await cur.fetchone()
        assert row is not None
        return _row_to_delivery_attempt(row)


    async def record_delivery(
        conn: aiosqlite.Connection,
        *,
        content_item_id: int,
        destination_id: int,
        route_id: int | None = None,
        success: bool,
        error: str | None = None,
    ) -> None:
        """Record a delivery outcome for one (content_item, destination) pair.

        This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
        sets status=delivered; on failure, increments attempt_count and records the
        error. Idempotent: repeated calls update the same row.
        """
        status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
        await conn.execute(
```

```
            """INSERT INTO delivery_attempts
                 (content_item_id, destination_id, route_id, status, attempt_count,
                  last_error, created_at, delivered_at)
               VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
                       CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
               ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
                 status=excluded.status,
                 attempt_count=delivery_attempts.attempt_count + 1,
                 last_error=excluded.last_error,
                 delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
                                   ELSE delivery_attempts.delivered_at END""",
            (
                content_item_id,
                destination_id,
                route_id,
                status.value,
                error,
                success,
                success,
            ),
        )
        await conn.commit()


    async def get_delivery_attempt(
```

4. user (2026-07-06T07:52:28.525Z)

```
    """Delivery: send matched videos to the user's Saved Messages.

    Three delivery modes (per-user, configured via FilterConfig.delivery_mode):

    - ``FORWARD``: forward the video itself (re-uploads to Saved Messages).
    - ``LINK``: send a text message with a t.me deep link + metadata summary.
    - ``BOTH``: forward the video AND send the link message.

    All modes wrap their Telethon calls with ``FloodWaitError`` handling (sleep
    ``seconds + jitter``, retry once) and mark the DB row ``forwarded=1`` on success.
    """

    from __future__ import annotations

    import asyncio
    import logging
    import random
    from collections.abc import Awaitable, Callable

    import aiosqlite
    from telethon import TelegramClient
    from telethon.errors import FloodWaitError
    from telethon.tl.types import Message

    from tg_video_filter import db
    from tg_video_filter.models import DeliveryMode

    log = logging.getLogger(__name__)

    # Extra jitter (seconds) added on top of the requested flood wait so we don't
    # retry at the exact boundary and risk re-triggering the limit.
    _FLOOD_JITTER_RANGE = (1.0, 5.0)


    # --- pure link-message formatting ---------------------------------------------


```

```
    def _format_duration(seconds: int | None) -> str:
        """Human-readable duration for the link summary."""
        if seconds is None:
            return "?"
        if seconds < 60:
            return f"{seconds}s"
        if seconds < 3600:
            return f"{seconds // 60}min"
        h = seconds // 3600
        m = (seconds % 3600) // 60
        return f"{h}h{m:02d}min" if m else f"{h}h"


    def _format_size(size_bytes: int | None) -> str:
        """Human-readable file size for the link summary."""
        if size_bytes is None:
            return "?"
        mib = size_bytes / (1024 * 1024)
        if mib >= 1024:
            return f"{mib / 1024:.1f}GB"
        return f"{mib:.0f}MB"


    def _format_height(height: int | None) -> str:
        """Resolution label: 1080 -> 1080p, None -> ?."""
        if height is None:
            return "?"
        return f"{height}p"


    def build_link_text(
        *,
        message_link: str | None,
        duration_s: int | None,
        height: int | None,
        size_bytes: int | None,
        channel_title: str | None,
    ) -> str:
        """Build the text message sent in LINK/BOTH mode.

        Format::

            ? 1080p · 5min · 480MB
            #ChannelName
            https://t.me/c/1234567890/42

        ``message_link`` may be None if the link can't be constructed (e.g. the
        channel is inaccessible); in that case the link line is omitted.
        """
        lines = [
            f"? {_format_height(height)} · {_format_duration(duration_s)} ·
{_format_size(size_bytes)}"
        ]
        if channel_title:
            # Sanitize: replace spaces in the title for a hashtag-like display.
            lines.append(f"#{channel_title.replace(' ', '_')}")
        if message_link:
            lines.append(message_link)
        return "\n".join(lines)


    # --- Telethon interaction (retry/flood logic) --------------------------------


    async def _send_with_retry(
```

```
        client: TelegramClient,
        send_callable: Callable[[], Awaitable[None]],
        video_id: int,
    ) -> bool:
        """Execute a send/forward callable with FloodWait handling. Returns success."""
        try:
            await send_callable()
        except FloodWaitError as e:
            wait = e.seconds + random.uniform(*_FLOOD_JITTER_RANGE)
            log.warning("flood_wait video=%s seconds=%s retrying_after=%.1f", video_id,
e.seconds, wait)
            await asyncio.sleep(wait)
            try:
                await send_callable()
            except Exception:
                log.exception("delivery_retry_failed video=%s", video_id)
                return False
        except Exception:
            log.exception("delivery_failed video=%s", video_id)
            return False
        return True


    async def forward_to_target(
        client: TelegramClient,
        message: Message,
        conn: aiosqlite.Connection,
        video_id: int,
        target: str = "me",
        *,
        destination_id: int | None = None,
    ) -> bool:
        """Forward ``message`` to ``target``, recording delivery state.

        ``target`` is a Telethon entity: ``"me"`` (Saved Messages), ``"@username"``,
        or a numeric chat id string. See ADR 0005.

        If ``destination_id`` is provided, a per-destination ``DeliveryAttempt``
        is written (ADR 0011 Track C). Otherwise falls back to the legacy no-op
        ``mark_forwarded``.
        """
        ok = await _send_with_retry(client, lambda: client.forward_messages(target,
message), video_id)
        if ok:
            if destination_id is not None:
                await db.record_delivery(
                    conn, content_item_id=video_id, destination_id=destination_id,
success=True
                )
            else:
                await db.mark_forwarded(conn, video_id)
            log.info("video_forwarded video=%s target=%s", video_id, target)
        elif destination_id is not None:
            await db.record_delivery(
                conn, content_item_id=video_id, destination_id=destination_id, success=False
            )
        return ok


    async def send_link_message(
        client: TelegramClient,
        text: str,
        conn: aiosqlite.Connection,
        video_id: int,
        target: str = "me",
```

```
        *,
        destination_id: int | None = None,
    ) -> bool:
        """Send a link/summary text message to ``target``, recording delivery state.

        ``target`` is a Telethon entity; see :func:`forward_to_target`.
        """
        ok = await _send_with_retry(client, lambda: client.send_message(target, text),
video_id)
        if ok:
            if destination_id is not None:
                await db.record_delivery(
                    conn, content_item_id=video_id, destination_id=destination_id,
success=True
                )
            else:
                await db.mark_forwarded(conn, video_id)
            log.info("link_sent video=%s target=%s", video_id, target)
        elif destination_id is not None:
            await db.record_delivery(
                conn, content_item_id=video_id, destination_id=destination_id, success=False
            )
        return ok


    async def deliver(
        client: TelegramClient,
        message: Message,
        conn: aiosqlite.Connection,
        video_id: int,
        mode: DeliveryMode,
        *,
        link_text: str | None = None,
        target: str = "me",
        destination_id: int | None = None,
    ) -> bool:
        """Dispatch delivery based on ``mode`` to ``target``.

        ``link_text`` is required for LINK/BOTH modes (built by the caller via
        :func:`build_link_text`). If it's None in LINK/BOTH mode, falls back to
        FORWARD so we never silently drop a matched video.

        ``target`` is a Telethon entity (``"me"``, ``"@username"``, chat id).
        See ADR 0005.

        ``destination_id`` is the FK to ``destinations.id`` (ADR 0011 Track C).
        When provided, a per-destination ``DeliveryAttempt`` is written.
        """
        if mode == DeliveryMode.LINK and link_text is not None:
            return await send_link_message(
                client, link_text, conn, video_id, target, destination_id=destination_id
            )
        if mode == DeliveryMode.BOTH and link_text is not None:
            fwd_ok = await forward_to_target(
                client, message, conn, video_id, target, destination_id=destination_id
            )
            link_ok = await send_link_message(
                client, link_text, conn, video_id, target, destination_id=destination_id
            )
            if not link_ok:
                log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
            if not fwd_ok:
                log.warning("forward_delivery_failed_in_both_mode video=%s", video_id)
            return fwd_ok or link_ok
        # FORWARD, or fallback when link_text is missing.
```

```
        if mode != DeliveryMode.FORWARD:
            log.warning(
                "link_mode_fallback_to_forward video=%s mode=%s link_text_missing=%s",
                video_id,
                mode.value,
                link_text is None,
            )
        return await forward_to_target(
            client, message, conn, video_id, target, destination_id=destination_id
        )


    __all__ = [
        "build_link_text",
        "deliver",
        "forward_to_target",
        "send_link_message",
    ]

```

5. assistant (2026-07-06T07:52:30.803Z)

Now let me look at the tests for both files, and the migration 004 code.

6. user (2026-07-06T07:52:31.439Z)

```
"""Tests for the link-message builder and delivery dispatcher.

The pure formatting (``build_link_text`` and its helpers) is fully unit-tested.
The ``deliver`` dispatcher is tested via fakes (FakeClient) since it wraps
Telethon calls.
"""

from __future__ import annotations

from unittest.mock import AsyncMock

import aiosqlite
import pytest
from telethon.errors import FloodWaitError

from tg_video_filter import db
from tg_video_filter.delivery import build_link_text, deliver
from tg_video_filter.models import DeliveryMode, Destination, User, VideoRecord

USER_ID = 1
CHANNEL_ID = 100
MSG_ID = 7


# --- build_link_text (pure formatting) ------------------------------------


def test_link_text_full_metadata() -> None:
    text = build_link_text(
        message_link="https://t.me/c/123/42",
        duration_s=300,
        height=1080,
        size_bytes=500 * 1024 * 1024,
        channel_title="Movie Channel",
    )
    assert "? 1080p · 5min · 500MB" in text
    assert "#Movie_Channel" in text
    assert "https://t.me/c/123/42" in text

```

```
40
41      def test_link_text_no_link() -> None:
42          """When message_link is None (e.g. inaccessible channel), the link line is
omitted."""
43          text = build_link_text(
44              message_link=None,
45              duration_s=120,
46              height=720,
47              size_bytes=50 * 1024 * 1024,
48              channel_title="Clips",
49          )
50          assert "? 720p · 2min · 50MB" in text
51          assert "#Clips" in text
52          assert "https://t.me" not in text
53
54
55      def test_link_text_no_channel_title() -> None:
56          text = build_link_text(
57              message_link="https://t.me/c/1/1",
58              duration_s=3600,
59              height=2160,
60              size_bytes=2 * 1024 * 1024 * 1024,
61              channel_title=None,
62          )
63          assert "? 2160p · 1h · 2.0GB" in text
64          assert "https://t.me/c/1/1" in text
65
66
67      def test_link_text_all_none_metadata() -> None:
68          """All metadata fields None -> question marks, link still present."""
69          text = build_link_text(
70              message_link="https://t.me/c/1/1",
71              duration_s=None,
72              height=None,
73              size_bytes=None,
74              channel_title=None,
75          )
76          assert "? ? · ? · ?" in text
77          assert "https://t.me/c/1/1" in text
78
79
80      @pytest.mark.parametrize(
81          ("seconds", "expected"),
82          [
83              (30, "30s"),
84              (59, "59s"),
85              (60, "1min"),
86              (300, "5min"),
87              (3600, "1h"),
88              (7200, "2h"),
89              (3661, "1h01min"),
90          ],
91      )
92      def test_format_duration(seconds: int, expected: str) -> None:
93          from tg_video_filter.delivery import _format_duration
94
95          assert _format_duration(seconds) == expected
96
97
98      @pytest.mark.parametrize(
99          ("mib", "expected"),
100         [
101             (1, "1MB"),
102             (500, "500MB"),
103             (1024, "1.0GB"),
```

```
            (2048, "2.0GB"),
        ],
    )
    def test_format_size(mib: int, expected: str) -> None:
        from tg_video_filter.delivery import _format_size

        assert _format_size(mib * 1024 * 1024) == expected


    # --- deliver() dispatcher ------------------------------------------------


    class FakeClient:
        """Tracks forward_messages and send_message calls and their target args."""

        def __init__(self, side_effects: list[BaseException | None] | None = None) -> None:
            self._side_effects = list(side_effects or [])
            self.forward_calls = 0
            self.send_calls = 0
            # Record the first positional arg (the target entity) of each call.
            self.forward_targets: list[object] = []
            self.send_targets: list[object] = []

        async def forward_messages(self, *a, **kw):
            self.forward_calls += 1
            if a:
                self.forward_targets.append(a[0])
            if self._side_effects:
                effect = self._side_effects.pop(0)
                if effect is not None:
                    raise effect

        async def send_message(self, *a, **kw):
            self.send_calls += 1
            if a:
                self.send_targets.append(a[0])
            if self._side_effects:
                effect = self._side_effects.pop(0)
                if effect is not None:
                    raise effect


    @pytest.fixture
    async def db_with_video() -> tuple[aiosqlite.Connection, int]:
        c = await aiosqlite.connect(":memory:")
        await db._apply_schema(c)
        await db.upsert_user(c, User(id=USER_ID))
        inserted = await db.insert_video(
            c, VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=MSG_ID)
        )
        assert inserted.id is not None
        yield c, inserted.id
        await c.close()


    _SENTINEL_MSG = object()


    @pytest.fixture
    def _no_sleep(monkeypatch: pytest.MonkeyPatch) -> None:
        monkeypatch.setattr("tg_video_filter.delivery.asyncio.sleep", AsyncMock())


    async def test_deliver_forward_mode(db_with_video: tuple[aiosqlite.Connection, int]) ->
None:
```

```
        c, vid = db_with_video
        client = FakeClient()
        ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.FORWARD,
link_text=None)  # type: ignore[arg-type]
        assert ok is True
        assert client.forward_calls == 1
        assert client.send_calls == 0


    async def test_deliver_link_mode(db_with_video: tuple[aiosqlite.Connection, int]) ->
None:
        c, vid = db_with_video
        client = FakeClient()
        ok = await deliver(
            client,
            _SENTINEL_MSG,
            c,
            vid,
            DeliveryMode.LINK,
            link_text="? test link",  # type: ignore[arg-type]
        )
        assert ok is True
        assert client.forward_calls == 0
        assert client.send_calls == 1


    async def test_deliver_both_mode(db_with_video: tuple[aiosqlite.Connection, int]) ->
None:
        c, vid = db_with_video
        client = FakeClient()
        ok = await deliver(
            client,
            _SENTINEL_MSG,
            c,
            vid,
            DeliveryMode.BOTH,
            link_text="? test link",  # type: ignore[arg-type]
        )
        assert ok is True
        assert client.forward_calls == 1
        assert client.send_calls == 1


    async def test_deliver_link_without_text_falls_back_to_forward(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """LINK mode with no link_text -> falls back to FORWARD (never drops a match)."""
        c, vid = db_with_video
        client = FakeClient()
        ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK, link_text=None)
# type: ignore[arg-type]
        assert ok is True
        assert client.forward_calls == 1
        assert client.send_calls == 0


    async def test_deliver_flood_then_retry(
        db_with_video: tuple[aiosqlite.Connection, int], _no_sleep: None
    ) -> None:
        c, vid = db_with_video
        client = FakeClient(side_effects=[FloodWaitError(request=None, capture=2), None])
        ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.FORWARD,
link_text=None)  # type: ignore[arg-type]
        assert ok is True
        assert client.forward_calls == 2  # initial + 1 retry
```

```
228
229
230     async def test_deliver_succeeds_without_forwarded_flag_post_adr_0011(
231         db_with_video: tuple[aiosqlite.Connection, int],
232     ) -> None:
233         """ADR 0011: delivery succeeds but ``forwarded`` stays False (no-op shim).
234
235         Post-migration-003 there is no ``forwarded`` column on ``content_items``;
236         per-destination delivery state moves to ``delivery_attempts`` in Track C.
237         For Phase 1 ``mark_forwarded`` is a no-op, so the projected
238         ``VideoRecord.forwarded`` is always False ? but delivery still returns True.
239         """
240         c, vid = db_with_video
241         client = FakeClient()
242         ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK,
link_text="test")  # type: ignore[arg-type]
243         assert ok is True
244         row = await db.get_video(c, USER_ID, CHANNEL_ID, MSG_ID)
245         assert row is not None
246         # forwarded is always False post-ADR-0011 (the column is dropped; delivery
247         # state moves to Track C). The match is still recorded via `matched`.
248         assert row.forwarded is False
249
250
251     async def test_deliver_with_destination_id_records_delivery_attempt(
252         db_with_video: tuple[aiosqlite.Connection, int],
253     ) -> None:
254         """When destination_id is provided, record_delivery writes a DeliveryAttempt (Track
C)."""
255         c, vid = db_with_video
256         # Create a destination row for the test
257         dest = await db.upsert_destination(
258             c, Destination(account_id=USER_ID, provider="telegram", target="me")
259         )
260         assert dest.id is not None
261         client = FakeClient()
262         await deliver(
263             client,
264             _SENTINEL_MSG,
265             c,
266             vid,
267             DeliveryMode.LINK,
268             link_text="test",
269             target="me",
270             destination_id=dest.id,  # type: ignore[arg-type]
271         )
272         # With destination_id, a DeliveryAttempt is written.
273         attempt = await db.get_delivery_attempt(c, vid, dest.id)
274         assert attempt is not None
275         assert attempt.status.value == "delivered"
276         assert attempt.attempt_count == 1
277
278
279     async def test_both_mode_logs_warning_on_link_failure(
280         db_with_video: tuple[aiosqlite.Connection, int],
281         caplog: pytest.LogCaptureFixture,
282     ) -> None:
283         """When BOTH mode's link send fails, a warning is logged."""
284         c, vid = db_with_video
285         # First call (forward) succeeds; second (send_message) raises.
286         client = FakeClient(side_effects=[None, RuntimeError("link fail")])
287         await deliver(
288             client,
289             _SENTINEL_MSG,
290             c,
```

```
            vid,
            DeliveryMode.BOTH,
            link_text="test",  # type: ignore[arg-type]
        )
        assert "link_delivery_failed_in_both_mode" in caplog.text


    async def test_link_mode_fallback_to_forward_is_logged(
        db_with_video: tuple[aiosqlite.Connection, int],
        caplog: pytest.LogCaptureFixture,
    ) -> None:
        """LINK mode with no link_text falls back to FORWARD and logs a warning."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK, link_text=None)  #
type: ignore[arg-type]
        assert "link_mode_fallback_to_forward" in caplog.text


    # --- delivery target (ADR 0005) ---------------------------------------------


    async def test_deliver_defaults_to_saved_messages(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """Omitting target uses 'me' (Saved Messages) ? the v1.1 default."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.FORWARD, link_text=None)
# type: ignore[arg-type]
        assert client.forward_targets == ["me"]


    async def test_deliver_forward_to_channel_username(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """FORWARD mode routes to an @username channel."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(
            client,
            _SENTINEL_MSG,  # type: ignore[arg-type]
            c,
            vid,
            DeliveryMode.FORWARD,
            link_text=None,
            target="@my-feed",
        )
        assert client.forward_targets == ["@my-feed"]
        assert client.send_calls == 0


    async def test_deliver_link_to_numeric_chat_id(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """LINK mode routes to a numeric chat id (private channel)."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(
            client,
            _SENTINEL_MSG,  # type: ignore[arg-type]
            c,
            vid,
            DeliveryMode.LINK,
            link_text="? test",
```

```
            target="-1001234567890",
        )
        assert client.send_targets == ["-1001234567890"]
        assert client.forward_calls == 0


    async def test_deliver_both_to_channel(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """BOTH mode routes both the forward and the link to the target."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(
            client,
            _SENTINEL_MSG,  # type: ignore[arg-type]
            c,
            vid,
            DeliveryMode.BOTH,
            link_text="? test",
            target="@curated",
        )
        assert client.forward_targets == ["@curated"]
        assert client.send_targets == ["@curated"]


    # --- FloodWait retry exhaustion (coverage improvement for delivery.py) --------


    async def test_flood_wait_retry_exhaustion(
        db_with_video: tuple[aiosqlite.Connection, int], _no_sleep: None
    ) -> None:
        """When both initial send AND retry raise FloodWaitError, delivery returns False."""
        c, vid = db_with_video
        client = FakeClient(
            side_effects=[
                FloodWaitError(request=None, capture=2),
                FloodWaitError(request=None, capture=5),
            ]
        )
        ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.FORWARD,
link_text=None)  # type: ignore[arg-type]
        assert ok is False
        assert client.forward_calls == 2  # initial + 1 retry, both failed


    async def test_flood_wait_retry_then_generic_failure(
        db_with_video: tuple[aiosqlite.Connection, int], _no_sleep: None
    ) -> None:
        """FloodWait handled, then generic Exception on retry ? returns False."""
        c, vid = db_with_video
        client = FakeClient(
            side_effects=[FloodWaitError(request=None, capture=1), RuntimeError("boom")]
        )
        ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.FORWARD,
link_text=None)  # type: ignore[arg-type]
        assert ok is False
        assert client.forward_calls == 2  # initial (flood) + 1 retry (exception)

```

7. user (2026-07-06T07:52:32.128Z)

```
    async def get_delivery_attempt(
        conn: aiosqlite.Connection, content_item_id: int, destination_id: int
    ) -> DeliveryAttempt | None:
        """Read a delivery attempt row by its natural key (idempotency boundary)."""
```

```
1403        conn.row_factory = aiosqlite.Row
1404        async with conn.execute(
1405            "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id =
?",
1406            (content_item_id, destination_id),
1407        ) as cur:
1408            row = await cur.fetchone()
1409        return _row_to_delivery_attempt(row) if row else None
1410
1411
1412    # --- legacy Channel shim (backwards-compatible view over sources) ------------
1413
1414
1415    async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1416        """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1417
1418        ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1419        reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1420        New code should call :func:`upsert_source` directly.
1421        """
1422        src = await upsert_source(
1423            conn,
1424            Source(
1425                account_id=channel.user_id,
1426                provider="telegram",
1427                provider_source_id=str(channel.channel_id),
1428                title=channel.title,
1429                kind="channel",
1430                watched=True,  # back-compat: pre-ADR-0011 channels were all watched
1431                seen_first_at=channel.seen_first_at,
1432            ),
1433        )
1434        return _source_to_channel(src)
1435
1436
1437    async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:
1438        """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
```

8. assistant (2026-07-06T07:52:35.510Z)

Now let me look at the migration 004 code and its tests.

9. user (2026-07-06T07:52:37.143Z)

```
668:async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
```

10. user (2026-07-06T07:52:38.418Z)

```
tests\test_delivery.py:254:    """When destination_id is provided, record_delivery writes a
DeliveryAttempt (Track C)."""
tests\test_db.py:1494:async def test_record_delivery_success(conn: aiosqlite.Connection) ->
None:
tests\test_db.py:1495:    """record_delivery with success=True sets status=delivered."""
tests\test_db.py:1515:    await db.record_delivery(conn, content_item_id=item.id,
destination_id=dest.id, success=True)
tests\test_db.py:1523:async def test_record_delivery_failure(conn: aiosqlite.Connection) ->
None:
tests\test_db.py:1524:    """record_delivery with success=False sets status=failed + error."""
tests\test_db.py:1544:    await db.record_delivery(
tests\test_db.py:1557:async def test_record_delivery_is_idempotent(conn: aiosqlite.Connection)
-> None:
tests\test_db.py:1558:    """Repeated record_delivery updates the same row; attempt_count
increments."""
```

```
tests\test_db.py:1578:    await db.record_delivery(
tests\test_db.py:1581:    await db.record_delivery(conn, content_item_id=item.id,
destination_id=dest.id, success=True)
tests\test_db.py:1591:async def test_migration_004_backfills_destinations_and_attempts(tmp_path:
object) -> None:
tests\test_db.py:1645:async def test_migration_004_is_idempotent(tmp_path: object) -> None:
```

11. assistant (2026-07-06T07:52:42.982Z)

`record_delivery` failure path IS tested at test_db.py:1523. Let me read the migration 004 code and its tests fully.

12. user (2026-07-06T07:52:43.993Z)

```
668     async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
669         """Backfill default destinations and delivery state (ADR 0011, Track C).
670
671         Creates a default destination row for each account that has a
672         ``filter_config_deprecated.delivery_target`` value, then creates a default
673         routing rule (source ? default policy ? destination) for each source.
674         Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
675         table (one ``delivered`` attempt per previously-forwarded video to the
676         account's default destination).
677
678         Idempotent: skips accounts whose default destination already exists.
679         """
680         # Gate: if destinations already have data, this migration has run.
681         async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
682             if (await cur.fetchone())[0] > 0:
683                 return
684
685         if not await _table_exists(conn, "filter_config_deprecated"):
686             return
687
688         # 1. Backfill default destinations from filter_config_deprecated.
689         #    Parse the JSON delivery_target, defaulting to "me".
690         await conn.execute(
691             """
692             INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
is_output, created_at)
693             SELECT user_id, 'telegram',
694                    COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),
695                    CASE
696                      WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') =
'me'
697                        THEN 'saved'
698                      ELSE 'channel'
699                    END,
700                    0,
701                    strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
702             FROM filter_config_deprecated
703             """
704         )
705
706         # 2. Create a default routing rule per source (source ? default policy ? default
destination).
707         await conn.execute(
708             """
709             INSERT OR IGNORE INTO routing_rules
710                 (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
711             SELECT s.account_id, s.id, p.id, d.id, 1, 0,
712                    strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
713             FROM sources s
714             JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
```

```
            JOIN destinations d ON d.account_id = s.account_id
            WHERE d.target = (
                SELECT COALESCE(
                    json_extract(fcd.config_json, '$.delivery_target'), 'me'
                )
                FROM filter_config_deprecated fcd
                WHERE fcd.user_id = s.account_id
                LIMIT 1
            )
            """
        )

        # 3. Backfill delivery_attempts from previously-forwarded videos.
        await conn.execute(
            """
            INSERT OR IGNORE INTO delivery_attempts
                (content_item_id, destination_id, route_id, status, attempt_count,
                 created_at, delivered_at)
            SELECT ci.id, d.id, rr.id, 'delivered', 1,
                   vd.seen_at, vd.seen_at
            FROM videos_deprecated vd
            JOIN content_items ci
              ON ci.account_id = vd.user_id
             AND ci.provider = 'telegram'
             AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
            JOIN sources sc
              ON sc.id = ci.source_id
             AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
            JOIN destinations d ON d.account_id = vd.user_id
            JOIN routing_rules rr
              ON rr.account_id = vd.user_id
             AND rr.source_id = sc.id
             AND rr.destination_id = d.id
            WHERE vd.forwarded = 1
            """
        )


    async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
        """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
        async with conn.execute(
            "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
        ) as cur:
            return await cur.fetchone() is not None


    # --- helpers ---------------------------------------------------------------


    def _now() -> datetime:
        """Timezone-aware UTC 'now', used for policy timestamps."""
        return datetime.now(UTC)


    def _safe_int_cast(value: object, field_name: str) -> int:
        """Cast a provider_source_id / provider_message_id / document_id to int.

        All existing data is Telegram ? these ids are numeric strings. Non-Telegram
        providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
        shapes (which carry Telegram-specific int ids); this function raises a clear
        ``ValueError`` rather than letting a cryptic ``int()`` crash propagate
        (ADR 0011 provider-neutrality guard ? review finding #5).
        """
        try:
            return int(value)  # type: ignore[arg-type]
```

```
780         except (ValueError, TypeError) as exc:
781             raise ValueError(
782                 f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
"
783                 f"types are Telegram-specific; use ContentItem/Source directly for "
784                 f"non-Telegram providers (ADR 0011)."
785             ) from exc
786 
787 
788     def _row_to_account(row: aiosqlite.Row) -> Account:
789         return Account(
790             id=row["id"],
791             telegram_id=row["telegram_id"],
792             provider=row["provider"],
793             display_name=row["display_name"],
794             created_at=row["created_at"],  # stored as ISO string; pydantic parses
795         )
796 
797 
798     def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
799         return TelegramSession(
800             account_id=row["account_id"],
801             phone=row["phone"],
802             session_path=row["session_path"],
803             enabled=bool(row["enabled"]),
804             created_at=row["created_at"],
805             last_login_at=row["last_login_at"],
806         )
807 
```

13. user (2026-07-06T07:52:44.553Z)

```
1470         assert fetched.kind == "saved"
1471 
1472 
1473     async def test_upsert_destination_conflict_updates(conn: aiosqlite.Connection) -> None:
1474         """Re-upsert with same (account, provider, target) updates kind/title."""
1475         await _seed_user(conn)
1476         await db.upsert_destination(
1477             conn,
1478             Destination(account_id=USER_ID, provider="telegram", target="@ch",
kind="channel"),
1479         )
1480         updated = await db.upsert_destination(
1481             conn,
1482             Destination(
1483                 account_id=USER_ID,
1484                 provider="telegram",
1485                 target="@ch",
1486                 kind="channel",
1487                 title="My Feed",
1488             ),
1489         )
1490         assert updated.title == "My Feed"
1491         assert updated.kind == "channel"
1492 
1493 
1494     async def test_record_delivery_success(conn: aiosqlite.Connection) -> None:
1495         """record_delivery with success=True sets status=delivered."""
1496         await _seed_user(conn)
1497         src = await db.upsert_source(
1498             conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
1499         )
1500         dest = await db.upsert_destination(
1501             conn, Destination(account_id=USER_ID, provider="telegram", target="me")
```

```
        )
        assert src.id is not None and dest.id is not None
        item = await db.insert_content_item(
            conn,
            ContentItem(
                account_id=USER_ID,
                source_id=src.id,
                provider="telegram",
                provider_message_id="99",
                dedup_key="tg:msg:1:99",
            ),
        )
        assert item.id is not None
        await db.record_delivery(conn, content_item_id=item.id, destination_id=dest.id,
success=True)
        attempt = await db.get_delivery_attempt(conn, item.id, dest.id)
        assert attempt is not None
        assert attempt.status == DeliveryStatus.DELIVERED
        assert attempt.attempt_count == 1
        assert attempt.delivered_at is not None


    async def test_record_delivery_failure(conn: aiosqlite.Connection) -> None:
        """record_delivery with success=False sets status=failed + error."""
        await _seed_user(conn)
        src = await db.upsert_source(
            conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
        )
        dest = await db.upsert_destination(
            conn, Destination(account_id=USER_ID, provider="telegram", target="me")
        )
        assert src.id is not None and dest.id is not None
        item = await db.insert_content_item(
            conn,
            ContentItem(
                account_id=USER_ID,
                source_id=src.id,
                provider="telegram",
                provider_message_id="100",
                dedup_key="tg:msg:1:100",
            ),
        )
        assert item.id is not None
        await db.record_delivery(
            conn,
            content_item_id=item.id,
            destination_id=dest.id,
            success=False,
            error="FloodWait(2s)",
        )
        attempt = await db.get_delivery_attempt(conn, item.id, dest.id)
        assert attempt is not None
        assert attempt.status == DeliveryStatus.FAILED
        assert attempt.last_error == "FloodWait(2s)"


    async def test_record_delivery_is_idempotent(conn: aiosqlite.Connection) -> None:
        """Repeated record_delivery updates the same row; attempt_count increments."""
        await _seed_user(conn)
        src = await db.upsert_source(
            conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
        )
        dest = await db.upsert_destination(
            conn, Destination(account_id=USER_ID, provider="telegram", target="me")
        )
```

```
1566        assert src.id is not None and dest.id is not None
1567        item = await db.insert_content_item(
1568            conn,
1569            ContentItem(
1570                account_id=USER_ID,
1571                source_id=src.id,
1572                provider="telegram",
1573                provider_message_id="101",
1574                dedup_key="tg:msg:1:101",
1575            ),
1576        )
1577        assert item.id is not None
1578        await db.record_delivery(
1579            conn, content_item_id=item.id, destination_id=dest.id, success=False,
error="first"
1580        )
1581        await db.record_delivery(conn, content_item_id=item.id, destination_id=dest.id,
success=True)
1582        attempt = await db.get_delivery_attempt(conn, item.id, dest.id)
1583        assert attempt is not None
1584        assert attempt.status == DeliveryStatus.DELIVERED
1585        assert attempt.attempt_count == 2  # failed + succeeded
1586
1587
1588    # --- Track C: Migration 004 ------------------------------------------------
1589
1590
1591    async def test_migration_004_backfills_destinations_and_attempts(tmp_path: object) ->
None:
1592        """Migration 004 backfills default destinations from filter_config and
1593        creates delivery_attempts for previously-forwarded videos."""
1594        db_path = str(tmp_path) + "/mig004.db"  # type: ignore[operator]
1595        # Build a pre-ADR-0011 legacy DB with a user, channel, video, and filter_config.
1596        legacy_schema = """
1597        CREATE TABLE users (id INTEGER PRIMARY KEY, phone TEXT, enabled INTEGER NOT NULL
DEFAULT 1, created_at TEXT NOT NULL);
1598        CREATE TABLE channels (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT
NULL REFERENCES users(id), channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
UNIQUE (user_id, channel_id));
1599        CREATE TABLE videos (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL
REFERENCES users(id), channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, duration_s
INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT
NULL DEFAULT 0, forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL, UNIQUE (user_id,
channel_id, message_id));
1600        CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY REFERENCES users(id),
config_json TEXT NOT NULL);
1601        """
1602        async with aiosqlite.connect(db_path) as conn:
1603            await conn.executescript(legacy_schema)
1604            await conn.execute("INSERT INTO users VALUES
(111,'+1',1,'2024-01-01T00:00:00Z')")
1605            await conn.execute(
1606                "INSERT INTO channels (user_id, channel_id, title, seen_first_at) VALUES
(111, -1001, 'Ch', '2024-01-01T00:00:00Z')"
1607            )
1608            await conn.execute(
1609                "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width,
height, size_bytes, mime_type, matched, forwarded, seen_at) "
1610                "VALUES (111, -1001, 42, 60, 1920, 1080, 1000, 'video/mp4', 1, 1,
'2024-01-01T00:00:00Z')"
1611            )
1612            # filter_config with @my-feed delivery_target
1613            await conn.execute(
1614                'INSERT INTO filter_config (user_id, config_json) VALUES (111,
\'{"delivery_target":"@my-feed","delivery_mode":"forward"}\')'
```

```
1615            )
1616            await conn.commit()
1617
1618        await db.init_db(db_path)
1619
1620        async with aiosqlite.connect(db_path) as conn:
1621            # Destination backfilled from filter_config.
1622            async with conn.execute(
1623                "SELECT target, kind FROM destinations WHERE account_id=111"
1624            ) as cur:
1625                d = await cur.fetchone()
1626            assert d == ("@my-feed", "channel")
1627
1628            # Routing rule created (source ? default policy ? destination).
1629            async with conn.execute(
1630                "SELECT COUNT(*) FROM routing_rules WHERE account_id=111 AND enabled=1"
1631            ) as cur:
1632                assert (await cur.fetchone())[0] >= 1
1633
1634            # Delivery attempt backfilled for the forwarded video.
1635            async with conn.execute("SELECT status, attempt_count FROM delivery_attempts")
as cur:
1636                da = await cur.fetchone()
1637            assert da is not None
1638            assert da == ("delivered", 1)
1639
1640            # user_version bumped to latest migration.
1641            async with conn.execute("PRAGMA user_version") as cur:
1642                assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
1643
1644
1645    async def test_migration_004_is_idempotent(tmp_path: object) -> None:
1646        """Migration 004 is a no-op on second run (destinations already populated)."""
1647        db_path = str(tmp_path) + "/idem004.db"  # type: ignore[operator]
1648        await db.init_db(db_path)
1649        await db.init_db(db_path)  # second run
1650        async with aiosqlite.connect(db_path) as conn:
1651            async with conn.execute("PRAGMA user_version") as cur:
1652                assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
1653            assert await db._table_exists(conn, "destinations")
1654            assert await db._table_exists(conn, "routing_rules")
1655            assert await db._table_exists(conn, "delivery_attempts")
1656
```

14. assistant (2026-07-06T07:52:53.490Z)

Now I have a good picture. Let me examine the key concerns:

1. `upsert_delivery_attempt` ? appears to have no test/caller. Let me confirm.
2. The `record_delivery` idempotent UPSERT logic when the first attempt is `delivered` and second is `failed` ? the `delivered_at` handling.
3. Migration 004 edge cases: 'me' default (no delivery_target key), no filter_config account, multi-source/multi-account.

Let me check for `upsert_delivery_attempt` callers across the whole source, and look at `_migration_004`'s idempotency test more carefully (the idempotent test uses `init_db` on an empty DB ? no legacy tables, so migration returns early at the `filter_config_deprecated` check, not the destinations-count gate).

15. user (2026-07-06T07:52:53.840Z)

```
src\tg_video_filter\db.py:1320:async def upsert_delivery_attempt(
src\tg_video_filter\db.py:1756:    "upsert_delivery_attempt",
```

16. assistant (2026-07-06T07:52:58.765Z)

Confirmed: `upsert_delivery_attempt` is defined (db.py:1320) and exported in `__all__` (db.py:1756) but has zero callers and zero tests. Now let me verify the coverage situation and check a couple of behavioral concerns with a repro. Let me first check the `delivered_at` reset bug in `record_delivery`: on a delivered?failed transition, does `delivered_at` get preserved but status flip to failed? And confirm migration idempotent test doesn't actually exercise the destinations-count gate.

Let me check what `_MIGRATIONS` looks like and whether idempotent test with empty init actually creates destinations (it asserts tables exist but the second `init_db` short-circuits before the gate).

17. user (2026-07-06T07:53:00.443Z)

```
336:async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
353:async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
505:async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
668:async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
```

18. user (2026-07-06T07:53:04.420Z)

```
191-
192-    Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
193:    then runs any pending migrations via the ``user_version`` ladder.
194-    """
195-    async with aiosqlite.connect(db_path) as conn:
196:        await _apply_schema(conn)
197-
198-
199:async def _apply_schema(conn: aiosqlite.Connection) -> None:
200-    # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
201-    # and a busy timeout so concurrent access waits instead of failing immediately.
--
220-# --- migrations ------------------------------------------------------------
221-#
222:# Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
223-# Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
224-# the source of truth for *fresh* databases; the migrations below reconcile
--
226-#
227-# To add a new migration:
228:#   1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
229-#   2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
230-#   3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
231-#      ``init_db``, and asserts the new structure.
232-
233:_MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
234-
235-
--
240-    Callable[[aiosqlite.Connection], Awaitable[None]],
241-]:
242:    """Register a migration that brings the DB up to ``user_version == version``."""
243-
244-    def decorator(
245-        fn: Callable[[aiosqlite.Connection], Awaitable[None]],
246-    ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
247:        _MIGRATIONS.append((version, fn))
248-        return fn
249-
--
314-
315-async def _apply_migrations(conn: aiosqlite.Connection) -> None:
316:    """Run every registered migration whose version > current ``user_version``."""
```

```
317:    async with conn.execute("PRAGMA user_version") as cur:
318-        row = await cur.fetchone()
319-    current = int(row[0]) if row else 0
320-
321:    for version, fn in _MIGRATIONS:
322-        if version <= current:
323-            continue
324-        await fn(conn)
325:        await conn.execute(f"PRAGMA user_version = {version}")
326-
327-
--
516-    # Idempotency gate: if the legacy videos table is gone, this migration has
517-    # already run (or the DB is fresh and _SCHEMA's legacy tables were already
518:    # dropped by _apply_schema post-003). Nothing to backfill.
519-    if not await _table_exists(conn, "videos"):
520-        return
```

19. assistant (2026-07-06T07:53:11.224Z)

Now I understand the migration ladder. The idempotency in `_apply_migrations` is via `user_version`, so the migration function's own internal gate (destinations COUNT > 0) only matters if `init_db` is called twice ? but the second call short-circuits at the version check anyway. Actually the internal gate matters for a subtle re-entrancy case, but not tested directly.

Let me now run a repro to verify two behavioral concerns and confirm the migration edge cases (the 'me' default and no-filter_config account are NOT tested). Let me write a repro script.

20. assistant (2026-07-06T07:53:33.368Z)

```
import asyncio, aiosqlite, os, tempfile
from tg_video_filter import db
from tg_video_filter.models import Destination, DeliveryAttempt, DeliveryStatus

LEGACY = """
CREATE TABLE users (id INTEGER PRIMARY KEY, phone TEXT, enabled INTEGER NOT NULL DEFAULT 1,
created_at TEXT NOT NULL);
CREATE TABLE channels (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL REFERENCES
users(id), channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL, UNIQUE
(user_id, channel_id));
CREATE TABLE videos (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL REFERENCES
users(id), channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, duration_s INTEGER, width
INTEGER, height INTEGER, size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL, UNIQUE (user_id, channel_id,
message_id));
CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT
NOT NULL);
"""

async def main():
    d = tempfile.mkdtemp()
    p = os.path.join(d, "t.db")
    async with aiosqlite.connect(p) as conn:
        await conn.executescript(LEGACY)
        # user 111: NO delivery_target key (should default to 'me' / saved)
        await conn.execute("INSERT INTO users VALUES (111,'+1',1,'2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO channels (user_id, channel_id, title, seen_first_at)
VALUES (111,-1001,'Ch','2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO videos (user_id,channel_id,message_id,duration_s,width,h
eight,size_bytes,mime_type,matched,forwarded,seen_at) VALUES
(111,-1001,42,60,1920,1080,1000,'video/mp4',1,1,'2024-01-01T00:00:00Z')")
        await conn.execute('INSERT INTO filter_config (user_id, config_json) VALUES (111,
\'{"delivery_mode":"forward"}\')')
        # user 222: TWO channels (multi-source), delivery_target @feed
```

```
        await conn.execute("INSERT INTO users VALUES (222,'+2',1,'2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO channels (user_id, channel_id, title, seen_first_at)
VALUES (222,-2001,'A','2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO channels (user_id, channel_id, title, seen_first_at)
VALUES (222,-2002,'B','2024-01-01T00:00:00Z')")
        await conn.execute('INSERT INTO filter_config (user_id, config_json) VALUES (222,
\'{"delivery_target":"@feed"}\')')
        # user 333: NO filter_config at all, has a channel + forwarded video
        await conn.execute("INSERT INTO users VALUES (333,'+3',1,'2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO channels (user_id, channel_id, title, seen_first_at)
VALUES (333,-3001,'C','2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO videos (user_id,channel_id,message_id,duration_s,width,h
eight,size_bytes,mime_type,matched,forwarded,seen_at) VALUES
(333,-3001,99,60,1920,1080,1000,'video/mp4',1,1,'2024-01-01T00:00:00Z')")
        await conn.commit()

    await db.init_db(p)

    async with aiosqlite.connect(p) as conn:
        conn.row_factory = aiosqlite.Row
        print("=== destinations ===")
        async with conn.execute("SELECT account_id,target,kind FROM destinations ORDER BY
account_id") as c:
            for r in await c.fetchall(): print(dict(r))
        print("=== routing_rules count per account ===")
        async with conn.execute("SELECT account_id, COUNT(*) n FROM routing_rules GROUP BY
account_id ORDER BY account_id") as c:
            for r in await c.fetchall(): print(dict(r))
        print("=== delivery_attempts ===")
        async with conn.execute("SELECT content_item_id,destination_id,status FROM
delivery_attempts") as c:
            for r in await c.fetchall(): print(dict(r))

    # --- record_delivery delivered->failed: does delivered_at survive but status flip? ---
    print("=== record_delivery delivered then failed ===")
    c2 = await aiosqlite.connect(":memory:")
    await db._apply_schema(c2)
    from tg_video_filter.models import User, Source, ContentItem
    await db.upsert_user(c2, User(id=1))
    src = await db.upsert_source(c2, Source(account_id=1, provider="telegram",
provider_source_id="-1"))
    dest = await db.upsert_destination(c2, Destination(account_id=1, provider="telegram",
target="me"))
    item = await db.insert_content_item(c2, ContentItem(account_id=1, source_id=src.id,
provider="telegram", provider_message_id="1", dedup_key="k1"))
    await db.record_delivery(c2, content_item_id=item.id, destination_id=dest.id, success=True)
    await db.record_delivery(c2, content_item_id=item.id, destination_id=dest.id, success=False,
error="later fail")
    a = await db.get_delivery_attempt(c2, item.id, dest.id)
    print("status=", a.status, "attempt_count=", a.attempt_count, "delivered_at=",
a.delivered_at, "last_error=", a.last_error)
    await c2.close()

asyncio.run(main())
```

21. user (2026-07-06T07:53:33.372Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

22. assistant (2026-07-06T07:53:35.707Z)

```
The file doesn't exist yet; the error is spurious for a new path but let me just use a different
filename.
```

23. user (2026-07-06T07:53:57.426Z)

```
=== destinations ===
{'account_id': 111, 'target': 'me', 'kind': 'saved'}
{'account_id': 222, 'target': '@feed', 'kind': 'channel'}
=== routing_rules count per account ===
{'account_id': 111, 'n': 1}
{'account_id': 222, 'n': 2}
=== delivery_attempts ===
{'content_item_id': 1, 'destination_id': 1, 'status': 'delivered'}
=== record_delivery delivered then failed ===
status= DeliveryStatus.FAILED attempt_count= 2 delivered_at= 2026-07-06 07:53:57+00:00
last_error= later fail
```

24. assistant (2026-07-06T07:54:05.238Z)

Interesting results. Key observations:

1. **Migration 004: account 333 (no filter_config) has NO destination** ? so its forwarded video (message 99) got NO delivery_attempt backfilled. The migration silently drops delivery history for accounts without a filter_config row. Only account 111's video (destination_id=1) got an attempt. This is a real data-loss-on-upgrade behavior, but is it in-scope? Account 333 has a forwarded video but no filter_config. Whether that's realistic depends on the app ? but the test suite does NOT cover this case at all. This is a **missing-test finding** and potentially a latent migration bug. Let me note the migration itself: no destination ? no routing rule ? no attempt. The forwarded history is lost.

2. **`record_delivery` delivered?failed leaves `delivered_at` populated while `status=FAILED`** ? an inconsistent row (status=failed but delivered_at set). This is the reverse of the tested idempotent case (failed?delivered). Untested, and arguably a latent logic quirk, though defensible.

Let me confirm the coverage report to pin down exactly which lines are uncovered (upsert_delivery_attempt).

25. user (2026-07-06T07:54:12.262Z)

```
src\tg_video_filter\db.py                        315     19     46      6    93%   520, 649-652,
683, 686, 896, 1324-1355, 1439-1440, 1452, 1494, 1778
src\tg_video_filter\delivery.py                   92      4     38      4    94%   145, 152, 181,
224
TOTAL                                           1102    535    264     11    47%
```

26. assistant (2026-07-06T07:54:21.173Z)

Coverage confirms the concerns precisely:

- **db.py 1324-1355**: the entire `upsert_delivery_attempt` body is uncovered (dead/untested).
- **db.py 683, 686**: the two early-return gates in migration 004 (`destinations COUNT > 0` and `not filter_config_deprecated`) are uncovered ? the no-filter_config path and the internal idempotency gate are never exercised.
- **delivery.py 145**: `forward_to_target` success + `destination_id` ? `record_delivery(success=True)` ? uncovered. The only delivery-path destination_id test uses LINK mode (line 174/181 side), so the FORWARD-mode record path is untested.
- **delivery.py 152**: `forward_to_target` failure + `destination_id` ? `record_delivery(success=False)` ? uncovered. The FAILURE recording path through `deliver` is NOT tested at all.
- **delivery.py 181**: `send_link_message` failure + `destination_id` ? `record_delivery(success=False)` ? uncovered.
- **delivery.py 224**: BOTH-mode forward-failure warning branch ? uncovered.

Let me confirm line 145 vs 174: which record_delivery lines are hit. The existing destination_id test (test_delivery.py:251) uses LINK mode, so it hits send_link_message success (line 174), leaving forward_to_target's record lines (145/152) and send_link's failure (181) uncovered. Let

me verify delivery.py line numbers map as I think.

27. user (2026-07-06T07:54:22.186Z)

```
142         ok = await _send_with_retry(client, lambda: client.forward_messages(target,
message), video_id)
143         if ok:
144             if destination_id is not None:
145                 await db.record_delivery(
146                     conn, content_item_id=video_id, destination_id=destination_id,
success=True
147                 )
148             else:
149                 await db.mark_forwarded(conn, video_id)
150             log.info("video_forwarded video=%s target=%s", video_id, target)
151         elif destination_id is not None:
152             await db.record_delivery(
153                 conn, content_item_id=video_id, destination_id=destination_id, success=False
154             )
155         return ok
156
157
158     async def send_link_message(
159         client: TelegramClient,
160         text: str,
161         conn: aiosqlite.Connection,
162         video_id: int,
163         target: str = "me",
164         *,
165         destination_id: int | None = None,
166     ) -> bool:
167         """Send a link/summary text message to ``target``, recording delivery state.
168
169         ``target`` is a Telethon entity; see :func:`forward_to_target`.
170         """
171         ok = await _send_with_retry(client, lambda: client.send_message(target, text),
video_id)
172         if ok:
173             if destination_id is not None:
174                 await db.record_delivery(
175                     conn, content_item_id=video_id, destination_id=destination_id,
success=True
176                 )
177             else:
178                 await db.mark_forwarded(conn, video_id)
179             log.info("link_sent video=%s target=%s", video_id, target)
180         elif destination_id is not None:
181             await db.record_delivery(
182                 conn, content_item_id=video_id, destination_id=destination_id, success=False
183             )
184         return ok
185
186
```

28. assistant (2026-07-06T07:54:32.211Z)

Confirmed. My analysis is complete. Let me finalize the findings.

Summary of the coverage/test-quality gaps in this PR:

1. **`upsert_delivery_attempt` (db.py:1320-1355) is fully dead + untested** ? defined, exported in `__all__` (line 1756), zero callers, zero tests (coverage 1324-1355 uncovered). `record_delivery` is the actual UPSERT used everywhere.

2. **The `destination_id` FAILURE recording path through `deliver` is never tested** ?

`forward_to_target` failure (delivery.py:151-154) and `send_link_message` failure (delivery.py:180-183) with `destination_id` set both write `record_delivery(success=False)` and are entirely uncovered. The one destination_id test (test_delivery.py:251) only exercises LINK-mode success.

3. **The FORWARD-mode `destination_id` success path is untested** ? delivery.py:144-147 (the `record_delivery(success=True)` inside `forward_to_target`) is uncovered; only the LINK-mode success line (174) is hit.

4. **Migration 004: the "no filter_config account" case is untested AND drops delivery history** ? repro shows account 333 (forwarded video, no filter_config row) gets no destination, no routing rule, and no delivery_attempt. Coverage lines 683/686 (both early-return gates) are uncovered.

5. **BOTH-mode + destination_id masking untested** ? no test forwards-ok/link-fail with destination_id, so it is never verified that the failed leg records `record_delivery(success=False)` while `deliver` still returns True (delivery.py:224 uncovered).

Now let me submit.

29. user (2026-07-06T07:55:14.173Z)

Structured output provided successfully